FSDP-Style Sharded Layerwise Offload: Optimization Report

Addressing PCIe Contention in SGLang's Wan2.2 Video Generation Pipeline

Danny Liu

February 24, 2026

1. Problem Statement

SGLang PR #15511 introduces a **layerwise offloading** strategy for video diffusion models that achieves a 58% speedup on NVLink systems by eliminating blocking transfer spikes. However, on our 4x H100 PCIe cluster (ACES), the new offload is **5.1% slower** than the existing FSDP-based offload (641.1s vs 610.2s for 27-step denoising), while using 62% less VRAM (23 GB vs 61 GB).

Our nsys CUPTI-level profiling definitively identified the root cause: **PCIe bus contention** between asynchronous H2D weight prefetching and NCCL collective communication.

2. Root Cause: PCIe Contention (Confirmed)

2.1 The Mechanism

The layerwise offload transfers the **full model** (~700 MB/layer, 28 GB/step per GPU) from CPU to GPU via async H2D on a dedicated CUDA copy stream. On PCIe systems, this H2D traffic shares the bus with Ulysses all-to-all (NCCL), causing mutual bandwidth degradation.

The old FSDP offload transfers only **1/N of each layer** (~175 MB with 4 GPUs), then uses NCCL all-gather to reconstruct. FSDP's topology-aware scheduling minimizes bus contention.

2.2 nsys Evidence

Metric	New Offload	Old Offload	Ratio
H2D total volume	5,767 GB	618 GB	9.3x
H2D during NCCL windows	5,253 GB	0 GB	∞
H2D BW during NCCL	7,907 MB/s	N/A	–
H2D BW without NCCL	12,447 MB/s	11,141 MB/s	1.1x
BW degradation during NCCL	**–36.5%**	**None**	–

Table 1: nsys temporal overlap analysis showing PCIe contention

61% of the new offload's H2D transfers overlap temporally with NCCL operations, degrading H2D bandwidth by 36.5%. The old offload has **zero** H2D during NCCL because its blocking `.to()` serializes all transfers before compute begins.

2.3 Impact on NCCL Performance

Metric	New Offload	Old Offload	Delta
NCCL total time	963.4s	690.5s	+39.5%
NCCL avg latency	15.7ms	11.4ms	+37.7%
NCCL max latency	4,145ms	1,701ms	+143%
all_to_allv (210MB) mean	21.4ms	16.2ms	+32.1%
all_to_allv (5MB) mean	340μs	341μs	unchanged

Table 2: NCCL performance degradation from PCIe contention

The key observation: **small transfers are unaffected** (340 us both), while large transfers degrade 32–143%. This rules out software bugs and confirms bandwidth contention as the mechanism.

2.4 Why NVLink Doesn't Have This Problem

On NVLink systems, GPU-GPU communication (900 GB/s NVLink) and CPU-GPU transfers (128 GB/s PCIe) use **physically separate paths**. H2D and NCCL never compete. On PCIe, both share the same bus.

	NVLink (DGX/SXM)	PCIe (ACES)
GPU↔GPU path	NVLink (900 GB/s)	PCIe (128 GB/s)
CPU→GPU path	PCIe (128 GB/s)	PCIe (128 GB/s)
Paths shared?	No	**Yes**
Steady-state delta	0.02s/step	**3.0s/step**

Table 3: NVLink vs PCIe topology and its effect on performance

3. The Optimization: Sharded Layerwise Offload

3.1 Core Idea

Apply FSDP-style weight sharding to the layerwise offload paradigm: each GPU copies only **1/N of each layer** from CPU, then uses `dist.all_gather_into_tensor()` to reconstruct the full weights. This reduces per-GPU PCIe H2D traffic by Nx while retaining the layerwise offload's low VRAM and smooth transformer-switch advantages.

3.2 Design

Transfer Pipeline (per layer, per step)

1. **Shard H2D** (copy stream): GPU r copies elements $[r \cdot S, (r+1) \cdot S)$ from the CPU buffer, where $S = \lceil \text{numel}/N \rceil$. Transfer size: ∼175 MB (vs 700 MB baseline).
2. **Wait for H2D** (compute stream): `wait_event` ensures the shard is on GPU before proceeding.
3. **All-gather** (compute stream): `dist.all_gather_into_tensor(full_buf, shard)` reconstructs the complete layer weights. Runs on the compute stream, naturally serialized with Ulysses all-to-all.

4. **Restore weights**: Slice the gathered buffer using existing metadata offsets and swap `.data` pointers.
5. **Forward pass**: Standard layer computation with full weights available.
6. **Release**: Free GPU memory (same as baseline).

Key Design Decisions

Decision	Rationale
All-gather on compute stream	Avoids cross-stream NCCL coordination. Serializes naturally with Ulysses all-to-all, preventing deadlock.
Default process group	All ranks execute layers 0–39 in identical order. Dedicated group deferred to v2.
Padded CPU buffers	Ensures even division by world_size. At most $N - 1$ extra elements per buffer.
Graceful single-GPU degradation	Falls back to non-sharded if `torch.distributed` not initialized or `world_size=1`.
Mutually exclusive with pcie_aware	Both solve PCIe contention differently; combining is redundant.

Table 4: Design decisions and rationale

NCCL Ordering Safety

All-gather calls are safe because:

1. All ranks execute layers 0–39 in **identical order** (same model, same forward pass)
2. Materialize is called in the **pre-hook** (compute stream), after the previous layer's Ulysses all-to-all synchronizes all ranks
3. The default process group **serializes** all-gather with Ulysses all-to-all, preventing deadlock

3.3 Implementation

The optimization was implemented in two versions:

1. **Local sglang-src** (newer API): Full patch in `patches/sharded_offload.patch` modifying `layerwise_offload.py` and `server_args.py`
2. **ACES sglang 0.5.8** (production): Direct file replacement in `patches/aces_layerwise_offload.py` adapted to the older API

Code Changes Summary

Component	Change
`__init__`	Added `sharded` param, distributed init, `_pending_shards` and `_shard_info` dicts
`_initialize()`	Pad CPU buffers for even sharding, compute shard boundaries
`prefetch_layer()`	Sharded branch: H2D only local 1/N shard
`_materialize_sharded_layer()`	New method: all-gather + weight restoration
Pre-hook	Call materialize after wait_event when sharded
`release_layer()`	Pop pending shards
`server_args.py`	Added `-dit-offload-sharded` CLI flag + validation

Table 5: Implementation changes

CLI Usage

```
python -m sglang.launch_server \
    --dit-layerwise-offload \
    --dit-offload-sharded \
    --ulysses-degree 4 \
    ...
```

4. Expected Performance

4.1 Quantitative Predictions

Metric	Current (New Offload)	Sharded (Expected)
H2D per GPU per step	28 GB (700MB × 40)	7 GB (175MB × 40)
PCIe root complex pressure	4× baseline	1× baseline (matches FSDP)
All-gather overhead	0	∼280ms/step (40 × 7ms)
NCCL contention	+39.5% slowdown	Eliminated
Net per-step time	∼23.7s	∼22.0–22.5s
Peak VRAM	23 GB	∼23 GB (unchanged)
Step 19 spike	None	None

Table 6: Expected performance comparison (4× H100 PCIe)

4.2 Reasoning

The sharded offload reduces per-GPU H2D by 4x (175 MB/layer vs 700 MB/layer). This should:

- **Eliminate NCCL contention**: 4x less H2D traffic means PCIe bus pressure drops to FSDP-equivalent levels. The 39.5% NCCL slowdown should largely disappear.

- **Add all-gather overhead**: Each layer requires an all-gather of approximately 700 MB across 4 GPUs (5–8ms per layer, 280ms/step total). This runs on the compute stream, adding to the critical path.
- **Net improvement**: Eliminating the 1.1s/step contention overhead minus 0.28s/step all-gather cost yields approximately **0.8s/step net improvement**.

4.3 Comparison with All Strategies

	Old (FSDP)	**New (Layerwise)**	**Sharded (Expected)**
Per-step time	22.6s	23.7s	22.0–22.5s
Total denoising	610.2s	641.1s	∼594–608s
Step 19 spike	+27s	None	None
Peak VRAM	61 GB	23 GB	∼23 GB
H2D/GPU/step	7 GB	28 GB	7 GB

Table 7: Head-to-head comparison of all three strategies

If the sharded offload achieves the expected 22s/step, it would be the **best of all worlds**: faster than FSDP (no switch spike), 62% less VRAM, and equivalent PCIe efficiency.

5. Experiment Status

5.1 Current Benchmark

The sharded offload has been submitted as ACES job **1473071** (February 24, 2026). The SLURM script runs 3 iterations:

1. **Warmup** (torch.compile JIT compilation)
2. **Clean timing** (definitive benchmark data)
3. **Profiled** (torch.profiler trace for detailed analysis)

Configuration: 4 H100 PCIe GPUs, Ulysses degree 4, SageAttention, torch.compile, 27 denoising steps, 720x1280x81 frames.

5.2 Job History

Batch	**Jobs**	**Status**	**Key Result**
Batch 2b (Feb 15)	1454665–70	Partial	4-GPU clean timing obtained
Batch 3 (Feb 18)	1459995–00	Mixed	nsys profiles; 6-GPU OOM
Batch 4 (Feb 24)	**1473071**	**Pending**	**Sharded offload benchmark**

Table 8: Experiment history

5.3 Verification Plan

Upon job completion, we will compare:

1. **Per-step timing** (Run 2, clean): target 22.5s/step or less steady-state

2. **Peak VRAM**: should remain at approximately 23 GB
3. **Total denoising time**: target 610s or less (matching or beating FSDP)
4. **Step 19 behavior**: should remain spike-free
5. **NCCL performance** (if profiled): all-to-all latency should return to FSDP-equivalent levels

6. Summary

Aspect	Detail
Problem	Layerwise offload 5.1% slower on PCIe due to H2D/NCCL bus contention
Root cause	9.3× more H2D traffic, 61% overlaps with NCCL, 36.5% BW degradation
Optimization	FSDP-style 1/N weight sharding + all-gather in layerwise offload
Mechanism	4× less H2D per GPU, all-gather serialized with Ulysses on compute stream
Expected result	∼22s/step (vs 23.7s current), matching FSDP speed with 62% less VRAM
Status	Implemented, deployed to ACES, benchmark job 1473071 pending

Appendix A: Architecture Diagram

```
               SHARDED LAYERWISE OFFLOAD (per layer)
               =====================================

CPU (pinned):  [======= Full Layer Buffer (~700MB) =======]
                |         |         |         |
                v         v         v         v
               GPU0      GPU1      GPU2      GPU3
              [175MB]   [175MB]   [175MB]   [175MB]
                \         |         |         /
                 \        |         |        /
            dist.all_gather_into_tensor (compute stream)
                 /        |         |        \
                /         |         |         \
              [======= Full Layer (~700MB) =======]
              GPU0      GPU1      GPU2      GPU3


               ORIGINAL LAYERWISE OFFLOAD (per layer)
               ======================================

CPU (pinned):  [======= Full Layer Buffer (~700MB) =======]
                |         |         |         |
```

```
  v          v          v          v
 GPU0       GPU1       GPU2       GPU3
[700MB]    [700MB]    [700MB]    [700MB]
 (each GPU independently copies the full layer)
```

Appendix B: Files Modified

File	Purpose
`patches/sharded_offload.patch`	Combined patch for local sglang-src
`patches/aces_layerwise_offload.py`	ACES-compatible complete file (sglang 0.5.8)
`patches/apply_sharded_patch.sh`	Patch application script
`patches/revert_sharded_patch.sh`	Patch revert script
`run_sharded_offload.slurm`	ACES SLURM benchmark script
`sglang-src/.../layerwise_offload.py`	Core implementation (local)
`sglang-src/.../server_args.py`	CLI argument registration (local)